Information Coding and Transmission Laboratory

Table of Contents

Exp No.	Experiment Name	Page No.
1	To develop a program to simulate and analyze the characteristics of common communication noises.	02
2	To develop a program to analyze the characteristics of Random Noise under different Mean and Variance values.	08
3	To develop a program to simulate a Noiseless Binary Channel and analyze its data transmission performance.	13
4	To develop a program to calculate Binary Symmetric Channel (BSC) Capacity.	15
5	To develop a program to implement the Shannon–Fano Coding Algorithm.	21
6	To develop a program to implement the Huffman Coding Algorithm.	26

Experiment No. 01

Experiment Name: To develop a program to simulate and analyze the characteristics of common communication noises.

Objective

1. To study the effect of common communication noises on a signal.
2. To simulate different noises used in communication engineering.
3. To observe how noise changes signal waveform and quality.
4. To compare the nature of different noises using graphical analysis.
5. To calculate basic performance parameters such as mean, variance, and signal-to-noise ratio.

Theory

In communication systems, the integrity of transmitted signals is frequently compromised by unwanted random disturbances, collectively known as noise. A noisy channel introduces stochastic variations into the signal, leading to potential distortion and decoding errors. Given its inherent randomness, noise is typically modeled as a stochastic process in communication theory.

1. Communication Noise

Communication noise refers to any undesirable electrical or electromagnetic disturbance that interferes with the original message signal during its transmission, processing, or reception.

2. Common Types of Communication Noise

a) Thermal Noise

Thermal noise, also known as Johnson–Nyquist noise, originates from the random thermal motion of charge carriers within electrical conductors. It is ubiquitously present in electronic communication systems and is commonly modeled as Gaussian noise. The thermal noise power is:

$$N = k \cdot T \cdot B$$

Where:

- $k = 1.38 \times 10^{-23}$ J/K, Boltzmann constant
- $T =$ absolute temperature in Kelvin
- $B =$ bandwidth in Hz

b) Shot Noise
Shot noise arises from the discrete nature of electric charge carriers, manifesting as random fluctuations in current. It is particularly significant in semiconductor devices, diodes, and optical communication receivers. Shot noise power may be expressed through current variance:

$$\sigma^2 = 2qIB$$

Where:

- $q = 1.6 \times 10^{-19}$ C
- $I =$ average current
- $B =$ bandwidth

c) Atmospheric Noise
Atmospheric noise is a natural phenomenon caused by electrical discharges in the atmosphere, such as lightning and thunderstorms. It predominantly affects radio communication, especially at lower frequencies.
d) Impulse Noise
Impulse noise is characterized by sudden, high-amplitude, and short-duration spikes. Common sources include switching circuits, power lines, and external electromagnetic interference. In simulations, it is often modeled by superimposing large random spikes onto the signal.

3. Noisy Channel Concept

A noisy channel alters the transmitted signal, thereby degrading communication quality. If $s(t)$ represents the original transmitted signal and $n(t)$ denotes the additive noise, the received signal $r(t)$ can be expressed as:

$$r(t) = s(t) + n(t)$$

4. Statistical Parameters Used in Noise Analysis

Mean
For a discrete set of N noise samples, denoted as $x_1, x_2, \ldots, x_N$, the mean (μ) is calculated as:

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i$$

Variance
The variance (σ^2) for the same set of noise samples quantifies their spread around the mean and is given by:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2$$

Signal-to-Noise Ratio
The Signal-to-Noise Ratio (SNR) is a critical metric for assessing signal quality, defined as the ratio of signal power (P_s) to noise power (P_n):

$$\text{SNR} = \frac{P_s}{P_n}$$

Expressed in decibels (dB), the SNR is:

$$\text{SNR(dB)} = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

Where:

- P_s = signal power
- P_n = noise power

Algorithm

1. Start the program.
2. Generate a clean message signal, for example a sine wave.
3. Define sampling frequency and time duration.
4. Generate different types of noise one by one:
    - thermal/AWGN noise
    - shot-like random noise
    - atmospheric/random burst noise
    - impulse noise
5. Add each noise to the original signal.
6. Plot the original signal and noisy signal.
7. Compute mean and variance of each noise.
8. Compute signal power, noise power, and SNR.
9. Compare the waveforms and statistical values.
10. Display the result.
11. Stop the program.

Simulation Code

```
import numpy as np
import matplotlib.pyplot as plt
import os

# Create figures directory if it doesn't exist
if not os.path.exists('figures'):
    os.makedirs('figures')

print("Running Experiment 01: Common Communication Noises...")

```

```
# Parameters
fs = 1000  # Sampling frequency
t = np.arange(0, 1.0, 1/fs)  # Time vector
f = 5  # Frequency of signal

# 1. Original Signal
signal = np.sin(2 * np.pi * f * t)

# 2. Thermal / AWGN Noise
thermal_noise = np.random.normal(0, 0.2, len(t))

# 3. Shot Noise (Modeled as random fluctuations)
shot_noise = np.random.poisson(0.1, len(t)) - 0.1

# 4. Atmospheric Noise (Random bursts)
atmospheric_noise = np.zeros(len(t))
burst_indices = np.random.randint(0, len(t), 10)
atmospheric_noise[burst_indices] = np.random.normal(0, 1.5, 10)

# 5. Impulse Noise
impulse_noise = np.zeros(len(t))
impulse_indices = np.random.randint(0, len(t), 5)
impulse_noise[impulse_indices] = np.random.choice([-2, 2], 5)

# Plotting
plt.figure(figsize=(12, 10))

plt.subplot(3, 2, 1)
plt.plot(t, signal)
plt.title('Original Signal')
plt.grid(True)

# AWGN
noisy_awgn = signal + thermal_noise
plt.subplot(3, 2, 2)
plt.plot(t, noisy_awgn)
plt.title('Signal with AWGN')
plt.grid(True)

# Shot
noisy_shot = signal + shot_noise
plt.subplot(3, 2, 3)
plt.plot(t, noisy_shot)
plt.title('Signal with Shot Noise')
plt.grid(True)

# Atmospheric
noisy_atm = signal + atmospheric_noise
plt.subplot(3, 2, 4)
plt.plot(t, noisy_atm)
plt.title('Signal with Atmospheric Noise')
```

```
plt.grid(True)

# Impulse
noisy_impulse = signal + impulse_noise
plt.subplot(3, 2, 5)
plt.plot(t, noisy_impulse)
plt.title('Signal with Impulse Noise')
plt.grid(True)

plt.tight_layout()
plt.savefig('figures/exp01_signal_noise.png')
plt.show()
print("Experiment 01 completed. Figure saved.")
```

Simulation Output

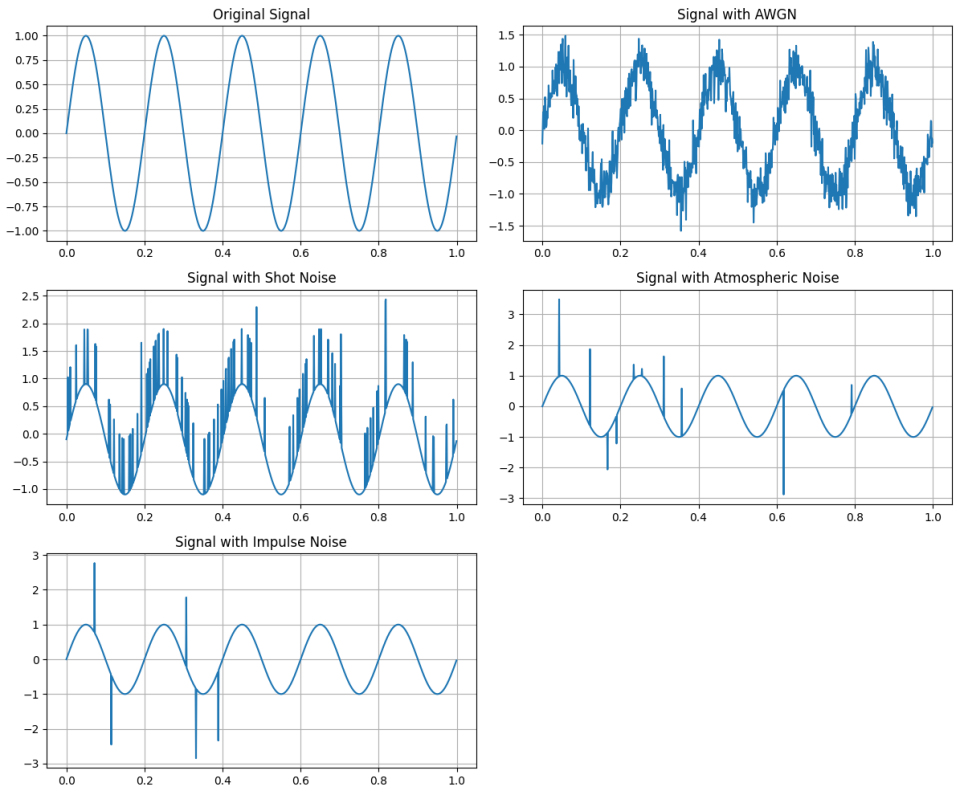

Figure 1: Different Types of Communication Channel Noises

Result & Discussion

The simulation results demonstrated that while the original signal was pristine, the introduction of various noise types led to distinct forms of distortion. Additive White Gaussian Noise (AWGN) resulted in small, continuous fluctuations. Impulse noise manifested as sudden, high-amplitude spikes. Thermal noise, consistent with Johnson–Nyquist principles, introduced a low-level background disturbance, and general random noise exhibited

irregular variations. Comparative analysis indicated that impulse noise had the most severe impact on signal integrity, whereas AWGN and thermal noise caused more gradual degradation. This experiment underscores the diverse ways in which noise impairs signal quality, highlighting the importance of robust system design in digital communication.

Experiment No. 02

Experiment Name: To develop a program to analyze the characteristics of Random Noise under different Mean and Variance values.

Objective

1. To generate random noise samples for different mean and variance values.
2. To observe the effect of mean on the position of the noise distribution.
3. To observe the effect of variance on the spread of the noise distribution.
4. To calculate important statistical parameters such as mean, variance, and standard deviation.
5. To compare different random noise cases using waveform and histogram analysis.

Theory

Random noise, an unpredictable signal that fluctuates irregularly over time, is a pervasive element in communication engineering. Its stochastic nature necessitates modeling as a probabilistic process, as its instantaneous value cannot be precisely determined. This experiment specifically investigates the statistical properties of random noise as its mean and variance are systematically altered. In practical simulations, Gaussian or normal noise models are frequently employed, given that numerous natural noise sources approximate a normal distribution.

1. Random Noise

A random noise sequence can be represented as $n(t)$, where $n(t)$ is a random function of time. For Gaussian noise, its amplitude distribution conforms to the normal probability density function.

2. Mean of Random Noise

The mean (μ) quantifies the average value or central tendency of the noise samples. For a set of N samples, $x_1, x_2, x_3, \ldots, x_N$, the mean is calculated as:

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i$$

Alternatively, in expanded form:

$$\mu = \frac{x_1 + x_2 + x_3 + \cdots + x_N}{N}$$

Interpretation of Mean

- If $\mu = 0$, the noise is centered around zero.
- If $\mu > 0$, the whole noise distribution shifts to the right.
- If $\mu < 0$, the distribution shifts to the left.

3. Variance of Random Noise

The variance (σ^2) provides a measure of the dispersion of noise samples around their mean, calculated as:

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$$

Interpretation of Variance

- Small variance means the noise samples stay close to the mean.
- Large variance means the noise samples are more scattered.

4. Standard Deviation

The standard deviation (σ), defined as the square root of the variance, is another metric for quantifying spread and is expressed in the same units as the noise amplitude:

$$\sigma = \sqrt{\sigma^2}$$

5. Gaussian Probability Density Function

For Gaussian random noise, the probability density function (PDF) is given by:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$

Where:

- μ = mean
- σ^2 = variance

6. Effect of Mean and Variance

Effect of Mean
A change in the mean shifts the entire distribution horizontally (left or right) without altering its spread, assuming constant variance.
Effect of Variance
Varying the variance directly impacts the width of the distribution:

- A smaller variance results in a narrower, more concentrated peak.
- A larger variance leads to a wider and flatter distribution.

Mathematical Relations

Sample Mean

$$\mu = \frac{1}{N}\sum x_i$$

Sample Variance

$$\sigma^2 = \frac{1}{N}\sum (x_i - \mu)^2$$

Algorithm

1. Start the program.
2. Select different values of mean and variance.
3. Generate random noise for each selected pair of mean and variance.
4. Plot the generated noise waveform.
5. Plot the histogram of the noise samples.
6. Calculate the experimental mean and variance from the generated samples.
7. Compare theoretical and simulated values.
8. Display all results.
9. Stop the program.

Simulation Code

```
import numpy as np
import matplotlib.pyplot as plt
import os

if not os.path.exists('figures'):
    os.makedirs('figures')

print("Running Experiment 02: Random Noise Analysis...")

# Parameters
N = 10000
mu_values = [0, 2, -2]
sigma_values = [0.5, 1.0, 2.0]

plt.figure(figsize=(12, 8))

# Effect of Mean (Constant Variance)
plt.subplot(2, 2, 1)
for mu in mu_values:
    noise = np.random.normal(mu, 1.0, N)
    plt.hist(noise, bins=50, alpha=0.6, label=f'Mean={mu}')
plt.title('Effect of Mean (Variance=1)')
plt.legend()
plt.grid(True)

# Effect of Variance (Constant Mean)
plt.subplot(2, 2, 2)
for sigma in sigma_values:
    noise = np.random.normal(0, sigma, N)
    plt.hist(noise, bins=50, alpha=0.6, label=f'Sigma={sigma}')
plt.title('Effect of Variance (Mean=0)')
```

```
plt.legend()
plt.grid(True)

# Waveform comparison
t = np.arange(N)
plt.subplot(2, 2, 3)
plt.plot(t[:500], np.random.normal(0, 0.5, N)[:500], label='Low Variance')
plt.plot(t[:500], np.random.normal(0, 2.0, N)[:500], label='High Variance')
plt.title('Waveform Comparison')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig('figures/exp02_random_noise_stats.png')
plt.show()
print("Experiment 02 completed. Figure saved.")
```

Simulation Output

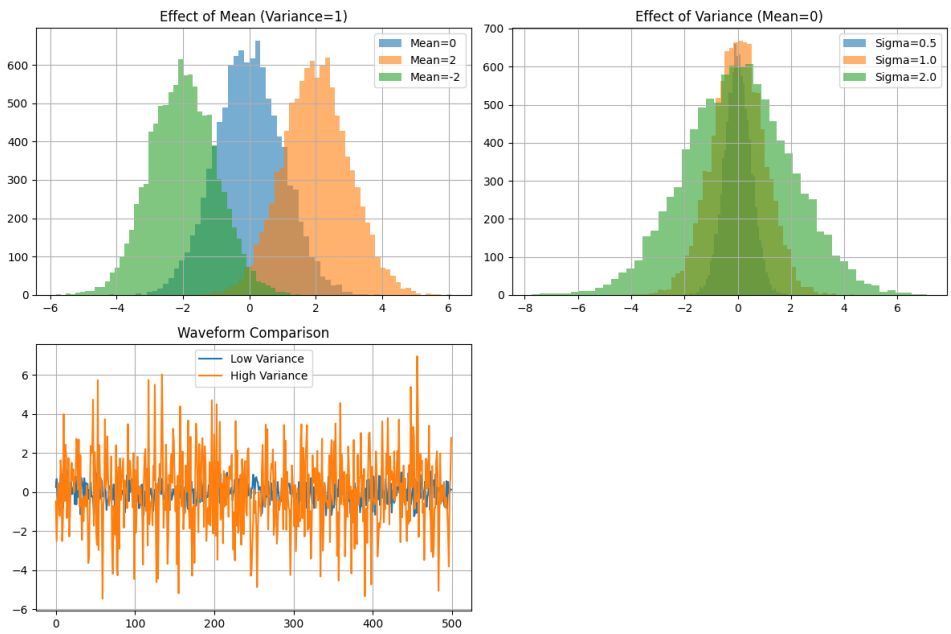

Figure 2: Effect of Mean and Variance on Random Noise

Observation

1. The generated noise followed the Gaussian distribution.
2. Changing the mean shifted the histogram along the x-axis.
3. Changing the variance changed the width and height of the histogram.

4. Simulated mean and variance are close to theoretical values, but not exactly equal due to randomness.

Result

A program was successfully developed to generate and analyze random noise under varying mean and variance values. Experimental observations confirmed that the mean dictates the central position of the noise distribution, while the variance governs its spread. This comprehensive analysis, employing both statistical parameters and graphical representations, effectively elucidated the characteristics of random noise.

Experiment No. 03

Experiment Name: To develop a program to simulate a Noiseless Binary Channel and analyze its data transmission performance.

Objective

1. To simulate a Noiseless Binary Channel using a computer program.
2. To observe how binary data is transmitted through a noiseless channel.
3. To verify that the output of a noiseless channel is identical to its input.
4. To calculate accuracy, bit error rate, and channel efficiency for the noiseless case.

Theory

A communication channel serves as the medium for transmitting information from a sender to a receiver. In a binary channel, the input alphabet is restricted to two symbols: 0 and 1. The Noiseless Binary Channel represents the most fundamental communication model, characterized by error-free transmission. Specifically, every transmitted bit is perfectly reproduced at the receiver (i.e., a transmitted 0 is received as a 0, and a transmitted 1 is received as a 1). According to *Elements of Information Theory*, this channel reproduces its binary input exactly at the output, enabling the reliable transmission of one bit per channel use, thus yielding a channel capacity of 1 bit.

Channel Representation

The noiseless binary channel is mathematically represented by the deterministic mappings:

- $0 \to 0$
- $1 \to 1$

Consequently, the output sequence is invariably identical to the input sequence.

Transition Probability Matrix

The transition probabilities for a noiseless binary channel are defined as:

- $P(Y = 0|X = 0) = 1$
- $P(Y = 1|X = 1) = 1$
- $P(Y = 1|X = 0) = 0$
- $P(Y = 0|X = 1) = 0$

Therefore, the channel transition matrix is the identity matrix:

$$P(Y|X) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

This matrix confirms the absence of noise, distortion, or bit-flipping during transmission.

Input-Output Relation

Given a transmitted bit sequence $X = \{x_1, x_2, x_3, \ldots, x_N\}$, the corresponding received sequence for a noiseless binary channel is $Y = \{y_1, y_2, y_3, \ldots, y_N\}$, where $y_i = x_i$ for all $i \in \{1, 2, \ldots, N\}$. Thus, $Y = X$, indicating perfect fidelity between the transmitted and received bits.

Channel Capacity

In a noiseless binary channel, exactly one bit of information is reliably transmitted per channel use. Consequently, the channel capacity (C) is:

$$C = 1 \text{ bit per channel use}$$

This represents the theoretical maximum capacity for any binary channel, constrained by its two-symbol input alphabet. This aligns with the principles outlined in *Elements of Information Theory*, which establishes the capacity of a noiseless binary channel as 1 bit.

Performance Parameters

1. Bit Error Rate (BER)
The Bit Error Rate (BER) is defined as the ratio of erroneously received bits to the total number of transmitted bits:

$$\text{BER} = \frac{\text{Number of erroneous bits}}{\text{Total number of transmitted bits}}$$

For a noiseless binary channel, the number of erroneous bits is zero. Therefore:

$$\text{BER} = \frac{0}{N} = 0$$

2. Accuracy
Transmission accuracy is defined as the percentage of correctly received bits relative to the total transmitted bits:

$$\text{Accuracy} = \frac{\text{Number of correctly received bits}}{\text{Total transmitted bits}} \times 100\%$$

Given that all bits are received without error in a noiseless channel:

$$\text{Accuracy} = \frac{N}{N} \times 100\% = 100\%$$

3. Efficiency
In noiseless transmission, every channel use successfully conveys information, resulting in a maximum channel efficiency (η) of 100%.

Mathematical Relations

Received Bit

$$Y_i = X_i$$

Algorithm

1. Start the program.
2. Input or generate a binary data sequence.
3. Pass the binary sequence through the noiseless binary channel.
4. Since the channel is noiseless, copy the transmitted sequence directly to the receiver.
5. Compare transmitted bits and received bits.
6. Count the number of errors.
7. Calculate BER and accuracy.
8. Display transmitted sequence, received sequence, and performance result.
9. Stop the program.

Simulation Code

```
import numpy as np
import matplotlib.pyplot as plt
import os

if not os.path.exists('figures'):
    os.makedirs('figures')

print("Running Experiment 03: Noiseless Binary Channel...")

N = 20
transmitted = np.random.randint(0, 2, N)

# Noiseless Channel Logic: Output = Input
received = transmitted.copy()

# Analysis
errors = np.sum(transmitted != received)
ber = errors / N
accuracy = 100.0

# Visualization
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.step(range(N), transmitted, where='mid', label='Transmitted',
    linewidth=2)
plt.step(range(N), received, where='mid', linestyle='--', label='
    Received')
plt.title(f'Noiseless Channel (BER={ber}, Accuracy={accuracy}%)')
plt.xlabel('Bit Index')
plt.ylabel('Value')
```

```
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.bar(['Transmitted', 'Received'], [np.sum(transmitted), np.sum(
    received)], color=['blue', 'orange'])
plt.title('Count of 1s')

plt.tight_layout()
plt.savefig('figures/exp03_noiseless_channel.png')
plt.show()
print("Experiment 03 completed. Figure saved.")
```

Simulation Output

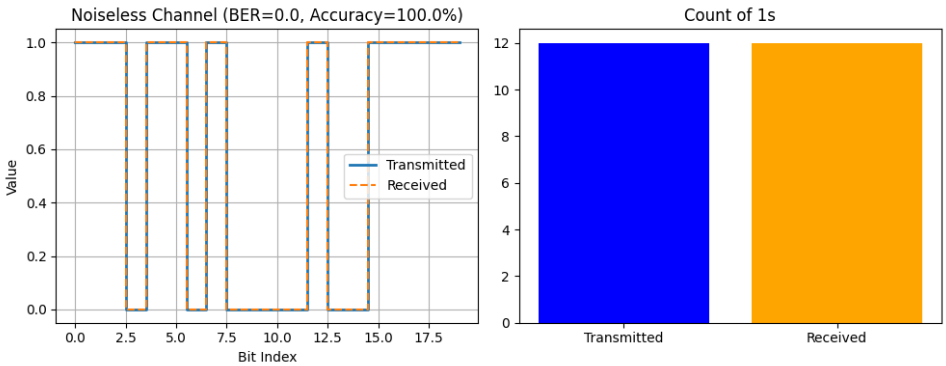

Figure 3: Data transmission in Noiseless Binary Channel

Experiment No. 04

Experiment Name: To develop a program to calculate Binary Symmetric Channel (BSC) Capacity.

Objective

1. To understand the concept of Binary Symmetric Channel (BSC).
2. To calculate the capacity of a BSC for a given crossover probability.
3. To observe how channel capacity changes with probability of error.
4. To verify the mathematical relation between probability of error and channel capacity.

Theory

A Binary Symmetric Channel (BSC) is a fundamental model in communication theory representing a binary channel where transmitted bits can be erroneously received with a crossover probability p. That means:

- 0 is received as 1 with probability p
- 1 is received as 0 with probability p

And:

- 0 is received correctly with probability $(1 - p)$
- 1 is received correctly with probability $(1 - p)$

As described in *Elements of Information Theory*, a BSC is a channel where the output matches the input with a probability of $1 - p$, and flips with a probability of p. Its capacity is formally defined as:

$$C = 1 + p \log_2 p + (1 - p) \log_2 (1 - p)$$

Transition Probabilities

- $P(Y = 0|X = 0) = 1 - p$
- $P(Y = 1|X = 1) = 1 - p$
- $P(Y = 1|X = 0) = p$
- $P(Y = 0|X = 1) = p$

Transition matrix:

$$P(Y|X) = \begin{bmatrix} 1 - p & p \\ p & 1 - p \end{bmatrix}$$

Channel Capacity Formula

The capacity of a BSC is elegantly expressed in terms of the binary entropy function, $H(p)$:

$$C = 1 - H(p)$$

where the binary entropy is given by:

$$H(p) = -p\log_2(p) - (1-p)\log_2(1-p)$$

Substituting the entropy function into the capacity formula yields:

$$C = 1 - [-p\log_2(p) - (1-p)\log_2(1-p)] = 1 + p\log_2(p) + (1-p)\log_2(1-p)$$

Important Cases

- If $p = 0$, then $C = 1$ bit. This means perfect transmission.
- If $p = 0.5$, then $C = 0$ bit. This means the output is completely random, so no useful information can be transmitted.
- If $0 < p < 0.5$, then $0 < C < 1$. So, higher error probability gives lower channel capacity.

Algorithm

1. Start the program.
2. Input the crossover probability p.
3. Calculate binary entropy $H(p)$.
4. Calculate channel capacity using: $C = 1 - H(p)$.
5. Display the value of capacity.
6. Repeat for different values of p.
7. Plot the graph of Capacity vs. Probability of Error.
8. Stop the program.

Simulation Code

```
import numpy as np
import matplotlib.pyplot as plt
import os

if not os.path.exists('figures'):
    os.makedirs('figures')

print("Running Experiment 04: BSC Capacity...")

```

```python
def binary_entropy(p):
    if p == 0 or p == 1:
        return 0
    return -p * np.log2(p) - (1 - p) * np.log2(1 - p)

# Range of error probabilities
p_values = np.linspace(0, 1, 100)
capacities = [1 - binary_entropy(p) for p in p_values]

# Visualization
plt.figure(figsize=(8, 6))
plt.plot(p_values, capacities, linewidth=2, color='green')
plt.title('Binary Symmetric Channel (BSC) Capacity')
plt.xlabel('Crossover Probability (p)')
plt.ylabel('Channel Capacity (C) bits/use')
plt.grid(True)

# Mark specific points
plt.scatter([0, 0.5, 1], [1, 0, 1], color='red', zorder=5)
plt.annotate('C=1, p=0', (0, 1), textcoords="offset points",
    xytext=(10, 5))
plt.annotate('C=0, p=0.5', (0.5, 0), textcoords="offset points",
    xytext=(10, 10))

plt.savefig('figures/exp04_bsc_capacity.png')
plt.show()
print("Experiment 04 completed. Figure saved.")
```

Simulation Output

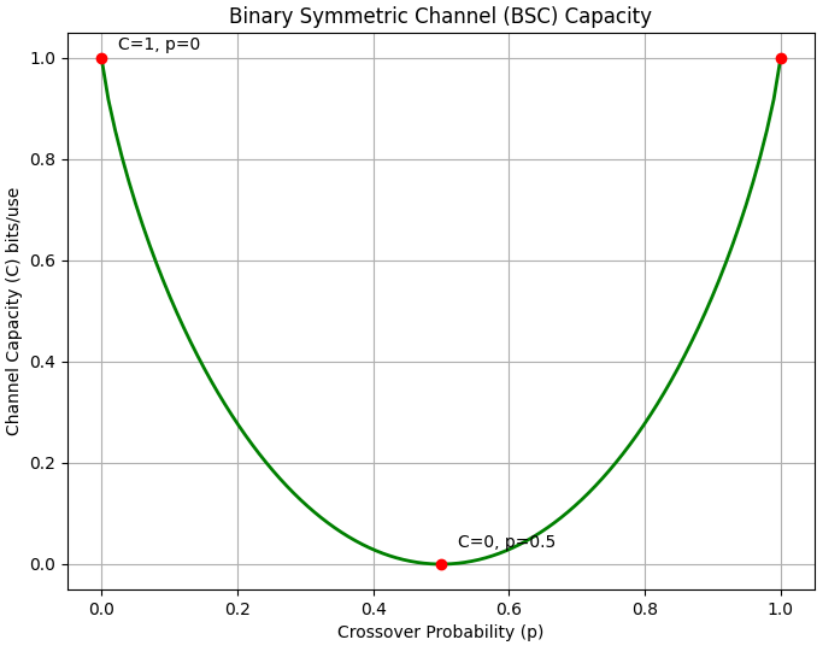

Figure 4: Binary Symmetric Channel (BSC) Capacity

Experiment No. 05

Experiment Name: To develop a program to implement the Shannon–Fano Coding Algorithm.

Objective

1. To understand the basic concept of source coding.
2. To implement the Shannon–Fano coding algorithm using a computer program.
3. To generate variable-length prefix codes for given source symbols.
4. To calculate entropy, average code length, and coding efficiency.
5. To study variable-length lossless compression.

Theory

Source coding is the process of converting a sequence of symbols into a binary code to achieve efficient data representation. The primary goals are to eliminate redundancy, facilitate data compression, and consequently reduce storage and transmission costs. The fundamental limit of this compression is defined by the entropy of the source. For a discrete random variable X, the entropy $H(X)$ quantifies the average information content or uncertainty and is calculated as:

$$H(X) = -\sum p(x) \log_2 p(x)$$

Entropy sets the theoretical minimum for the average number of bits required to encode a symbol from the source. Shannon–Fano coding is a variable-length, lossless data compression technique that aims to approach this limit. The algorithm operates by arranging symbols in descending order of their probabilities and then recursively partitioning them into two subgroups with nearly equal total probabilities. A '0' is assigned to one subgroup and a '1' to the other, with this process continuing until each symbol is isolated in its own subgroup, thereby generating a unique codeword. While related to Shannon–Fano–Elias coding in its reliance on probability ordering, Shannon–Fano coding provides an average code length that is close to the source entropy, though not always fully optimal like Huffman coding.

The core principle of Shannon–Fano coding, consistent with efficient source compression, is to assign shorter codewords to symbols with higher probabilities and longer codewords to those with lower probabilities.

Main Principle

If source symbols are: $S = \{s_1, s_2, s_3, \ldots, s_n\}$ with probabilities: $P = \{p_1, p_2, p_3, \ldots, p_n\}$, then the steps of Shannon–Fano coding are:

1. Sort symbols in descending order of probability.
2. Divide the set into two groups with nearly equal total probability.

3. Assign 0 to the first group and 1 to the second group.
4. Repeat the process recursively until one symbol remains in each group.

Mathematical Relations

Entropy:

$$H(X) = -\sum p(x) \log_2 p(x)$$

Average Code Length:

$$L = \sum p_i l_i$$

where p_i = probability of symbol i, l_i = length of codeword of symbol i.
Coding Efficiency: Coding efficiency is a measure of how effectively the source coding scheme utilizes the available bits, calculated as:

$$\text{Efficiency} = \frac{H}{L} \times 100\%$$

where H represents the entropy and L is the average code length.
Redundancy: Redundancy quantifies the difference between the average code length and the source entropy, indicating the inefficiency of the coding scheme. It can be expressed as:

$$\text{Redundancy} = 1 - \frac{H}{L}$$

Alternatively, in percentage form:

$$\text{Redundancy} = \left[1 - \frac{H}{L}\right] \times 100\%$$

Algorithm

1. Start the program.
2. Input the symbols and their probabilities.
3. Sort the symbols in descending order of probability.
4. Divide the symbols into two groups with nearly equal probability sum.
5. Assign 0 to the first group and 1 to the second group.
6. Repeat the same process for each subgroup recursively.
7. Continue until each symbol gets a unique codeword.
8. Calculate entropy.
9. Calculate average code length.
10. Calculate coding efficiency.
11. Display symbols, probabilities, codes, entropy, and efficiency.
12. Stop the program.

Simulation Code

```
import numpy as np
import matplotlib.pyplot as plt
import os

if not os.path.exists('figures'):
    os.makedirs('figures')

print("Running Experiment 05: Shannon-Fano Coding...")

# Input Data
symbols = ['s1', 's2', 's3', 's4', 's5']
probs = [0.4, 0.2, 0.2, 0.1, 0.1]

# Shannon-Fano Implementation
def shannon_fano(symbols, probs):
    # Sort by probability descending
    combined = list(zip(symbols, probs))
    combined.sort(key=lambda x: x[1], reverse=True)

    codes = {}

    def recursive_split(group):
        if len(group) == 1:
            return

        total_prob = sum([x[1] for x in group])
        half_prob = total_prob / 2
        current_sum = 0
        split_index = 0

        # Find split index
        for i in range(len(group)):
            current_sum += group[i][1]
            if current_sum >= half_prob:
                split_index = i + 1
                break

        # Safety for small groups
        if split_index == len(group):
            split_index = len(group) - 1

        group1 = group[:split_index]
        group2 = group[split_index:]

        # Assign '0' to group1, '1' to group2
        for sym, _ in group1:
            codes[sym] = codes.get(sym, '') + '0'
        for sym, _ in group2:
            codes[sym] = codes.get(sym, '') + '1'
```

```python

        recursive_split(group1)
        recursive_split(group2)

    recursive_split(combined)
    return codes, combined

codes, sorted_data = shannon_fano(symbols, probs)

# Calculations
entropy = -sum([p * np.log2(p) for p in probs])
avg_len = sum([probs[i] * len(codes[symbols[i]]) for i in range(len(symbols))])
efficiency = (entropy / avg_len) * 100

# Visualization
fig, ax = plt.subplots(figsize=(8, 4))
ax.axis('tight')
ax.axis('off')

table_data = []
for i, sym in enumerate(symbols):
    table_data.append([sym, probs[i], codes[sym], len(codes[sym])])

table = ax.table(cellText=table_data, colLabels=['Symbol', 'Probability', 'Code', 'Length'], loc='center', cellLoc='center')
table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1, 1.5)

plt.title(f'Shannon-Fano Coding\nEntropy: {entropy:.3f} bits, Avg Length: {avg_len:.3f} bits, Efficiency: {efficiency:.2f}%')
plt.savefig('figures/exp05_shannon_fano.png')
plt.show()
print("Experiment 05 completed. Figure saved.")
```

Simulation Output

Shannon-Fano Coding
Entropy: 2.122 bits, Avg Length: 2.200 bits, Efficiency: 96.45%

Symbol	Probability	Code	Length
s1	0.4	00	2
s2	0.2	01	2
s3	0.2	10	2
s4	0.1	110	3
s5	0.1	111	3

Figure 5: Shannon-Fano Coding for 5 Symbols

Observation

1. Symbols with higher probability got shorter codewords.
2. Symbols with lower probability got longer codewords.
3. Average code length was close to entropy.
4. Shannon–Fano coding achieved good compression efficiency.
5. The code is variable-length and lossless.

Experiment No. 06

Experiment Name: To develop a program to implement the Huffman Coding Algorithm.

Objective

1. To understand the basic concept of Huffman coding.
2. To implement the Huffman coding algorithm using a computer program.
3. To generate variable-length prefix codes for given source symbols.
4. To assign shorter codewords to more probable symbols and longer codewords to less probable symbols.
5. To calculate entropy, average code length, and coding efficiency.
6. To verify that Huffman code gives an optimal uniquely decodable code.

Theory

Source coding is a fundamental process in information theory aimed at representing source symbols using the minimum possible average number of bits, thereby achieving data compression without information loss. This process is crucial for reducing redundancy and minimizing storage and transmission costs. The entropy of a discrete random variable X, which quantifies the average uncertainty or information content, is defined as:

$$H(X) = -\sum p(x) \log_2 p(x)$$

Entropy also establishes the theoretical lower bound for the average number of bits required to encode the output of a source. Huffman coding is a widely used variable-length source coding technique that leverages symbol probabilities to assign shorter codewords to more probable symbols and longer codewords to less probable ones. A key characteristic of Huffman codes is their prefix-free nature, meaning no codeword is a prefix of another, which ensures unique decodability. Furthermore, Huffman coding is renowned for its optimality, as it minimizes the average codeword length for a given source distribution among all uniquely decodable codes, satisfying the Huffman bound:

$$H(X) \leq L_H < H(X) + 1$$

This method is a cornerstone of data compression and is considered an optimal prefix coding technique.

Main Principle of Huffman Coding

1. Arrange the symbols in ascending order of probability.
2. Select the two symbols with the smallest probabilities.
3. Combine them into a new node whose probability is the sum of the two probabilities.

4. Repeat the process until only one node remains.
5. Assign 0 and 1 to the branches of the tree.
6. Read the codeword of each symbol from the root to the leaf.

Consequently, Huffman coding effectively generates a prefix-free and optimal code.

Mathematical Relations

Entropy

$$H(X) = -\sum p(x) \log_2 p(x)$$

Average Code Length

$$L = \sum p_i l_i$$

where p_i = probability of symbol i, l_i = length of codeword of symbol i.
Coding Efficiency Coding efficiency measures the effectiveness of the Huffman coding scheme in compressing data, calculated as:

$$\text{Efficiency} = \frac{H}{L} \times 100\%$$

Redundancy Redundancy in Huffman coding indicates the extent to which the average codeword length exceeds the source entropy, expressed as:

$$\text{Redundancy} = \left[1 - \frac{H}{L}\right] \times 100\%$$

Huffman Bound

$$H(X) \leq L_H < H(X) + 1$$

Algorithm

1. Start the program.
2. Input the symbols and their probabilities.
3. Create a list of nodes with symbol and probability.
4. Sort the nodes in ascending order of probability.
5. Select the two nodes with the smallest probabilities.
6. Merge them into a new node.
7. Insert the new node into the list.
8. Repeat steps 4 to 7 until one node remains.
9. Traverse the tree and assign 0 to the left branch and 1 to the right branch.
10. Generate Huffman codewords for all symbols.
11. Calculate entropy.

12. Calculate average code length.
13. Calculate coding efficiency.
14. Display the result.
15. Stop the program.

Simulation Code

```
import numpy as np
import matplotlib.pyplot as plt
import heapq
import os

if not os.path.exists('figures'):
    os.makedirs('figures')

print("Running Experiment 06: Huffman Coding...")

# Input Data
symbols = ['a', 'b', 'c', 'd', 'e']
probs = [0.4, 0.2, 0.2, 0.1, 0.1]

# Huffman Implementation
class Node:
    def __init__(self, prob, symbol=None, left=None, right=None):
        self.prob = prob
        self.symbol = symbol
        self.left = left
        self.right = right

    def __lt__(self, other):
        return self.prob < other.prob

# Create priority queue
heap = [Node(p, s) for s, p in zip(symbols, probs)]
heapq.heapify(heap)

# Build Tree
while len(heap) > 1:
    left = heapq.heappop(heap)
    right = heapq.heappop(heap)
    merged = Node(left.prob + right.prob, left=left, right=right)
    heapq.heappush(heap, merged)

# Generate Codes
def generate_codes(node, prefix="", code_map={}):
    if node is None:
        return
    if node.symbol is not None:
```

```
        code_map[node.symbol] = prefix
        return
    generate_codes(node.left, prefix + "0", code_map)
    generate_codes(node.right, prefix + "1", code_map)
    return code_map

codes = generate_codes(heap[0], "", {})

# Calculations
entropy = -sum([p * np.log2(p) for p in probs])
avg_len = sum([probs[i] * len(codes[symbols[i]]) for i in range(len(symbols))])
efficiency = (entropy / avg_len) * 100

# Visualization
fig, ax = plt.subplots(figsize=(8, 4))
ax.axis('tight')
ax.axis('off')

table_data = []
for sym in symbols:
    idx = symbols.index(sym)
    table_data.append([sym, probs[idx], codes[sym], len(codes[sym])])

table = ax.table(cellText=table_data, colLabels=['Symbol', 'Probability', 'Code', 'Length'], loc='center', cellLoc='center')
table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1, 1.5)

plt.title(f'Huffman Coding\nEntropy: {entropy:.3f} bits, Avg Length: {avg_len:.3f} bits, Efficiency: {efficiency:.2f}%')
plt.savefig('figures/exp06_huffman_coding.png')
plt.show()
print("Experiment 06 completed. Figure saved.")
```

Simulation Output

Huffman Coding
Entropy: 2.122 bits, Avg Length: 2.200 bits, Efficiency: 96.45%

Symbol	Probability	Code	Length
a	0.4	11	2
b	0.2	01	2
c	0.2	00	2
d	0.1	101	3
e	0.1	100	3

Figure 6: Huffman Coding for 5 Symbols

Observation

1. Symbols with higher probability got shorter codewords.
2. Symbols with lower probability got longer codewords.
3. The Huffman code generated was prefix-free.
4. The average code length was very close to entropy.
5. Huffman coding showed high compression efficiency.